# Problem Framing

Most resume-matching systems do keyword overlap dressed up as AI. The JD explicitly warns against this: a candidate who lists every AI buzzword but whose title is “Marketing Manager” is not a fit, and a candidate who built a recommendation system at a product company **without** using the words “RAG” or “embeddings” **is** a fit.

We built a ranking system that reads career history for evidence of what someone actually did, not what vocabulary they used. Every design decision was driven by the JD’s explicit disqualifiers and ideal-profile signals, not by pattern-matching keywords.

# Architecture

Four-stage pipeline:

**1. JD Rules** — Disqualifiers and ideal-profile criteria encoded explicitly from the JD text, not re-derived by an LLM at runtime.
**2. Feature Extraction** — Shipped-system evidence, experience band, location fit, title-domain classification, disqualifier flags — all via precompiled word-boundary regex for both correctness and speed at 100K scale.
**3. Semantic Layer** — all-MiniLM-L6-v2 embeddings (precomputed offline) catch paraphrased fit the rules miss.
**4. Transparent Weighted Formula** — Fuses all signals into one score: 35% semantic, 25% shipped-evidence, 15% experience band, 10% location, –15% disqualifiers, × availability multiplier. No black box.

[Flow: JD Rules \u2192 Feature Extraction \u2192 Semantic Similarity \u2192 Weighted Formula \u2192 Availability Multiplier \u2192 Final Score]

# Why a Formula, Not a Learned Ranker

No labeled (JD, candidate, outcome) data exists for this task. Training LambdaMART or any learned ranker on self-generated labels would be circular and indefensible.

A hand-weighted formula is something we can recite and justify line-by-line, which matters directly for the Stage 5 interview:

**Semantic similarity (35%)** — Embedding cosine similarity catches semantic fit the keyword rules miss.
**Shipped-system evidence (25%)** — Counts ranking/search/recsys domain keywords and tech stack mentions in descriptions.
**Experience band (15%)** — Peaks at 6–8 years as specified in the ideal profile. Bonus for B-Tech + M-Tech/PhD.
**Location fit (10%)** — Noida, Pune, or explicitly relocatable.
**Disqualifier penalties (–15%)** — Consulting-only career, pure-research background, CV/speech without NLP, job-hopping, framework tourism.
**Availability multiplier (×)** — Immediate or 30-day joiners score higher.

# Caught Our Own False Positives

Three bugs found and fixed during development, each demonstrating the system distinguishes signal from noise rather than pattern-matching superficially:

**1. Naive substring matching** — A Marketing Manager who took online RAG/LangChain courses scored 0.8 shipped-evidence because “live” matched inside “delivered”. Fixed with word-boundary regex → 0.125.

**2. Off-domain titles in top 50** — Six titles (Graphic Designer ×3, Accountant, Customer Support, Sales Executive) ranked in the top 50 because their synthetic descriptions contained ranking/search keywords. Added title-domain classifier → 0 off-domain titles in top 50.

**3. Title classifier substring bug** — “ai” matched inside “Retail Analyst”, “search” matched inside “Research Analyst”, giving 0 penalty to clearly off-domain profiles. Fixed with same word-boundary approach → Retail Analyst now correctly gets 0.6 off-domain penalty.

# Verification Results

Run on CPU (4 cores, 16 GB RAM, no network) against full 100K candidate dataset:

- **Runtime:** 97.8 seconds (well within 300s budget)
- **Honeypot rate, top-100:** 0% (threshold: ≤10%)
- **Keyword-stuffer traps in top-100:** 0
- **Off-domain titles in top-50:** 0

Top 10 results: All legitimate ML/AI titles at product companies — RecSys Engineer at Wysa/Amazon/CRED, Sr. ML Engineer at Zomato, Sr. AI Engineer at Meta, NLP Engineer at Haptik, etc.

Verification suite runs on any output: honeypot check, keyword-stuffer detection, title-domain audit, runtime measurement.

# Compute & Reproducibility

Single reproduce command:

```
python rank.py --candidates ./candidates.jsonl --out ./submission.csv
```

Embedding precomputation (offline, exceeds 5 min) is separated from the timed ranking step. The ranking step loads cached embeddings and completes in under 2 minutes on CPU with **no network calls**.

**Pre-computation:** python precompute.py --candidates candidates.jsonl --out embeddings.npy

**Dependencies:** sentence-transformers, numpy, scikit-learn

**Sandbox:** Streamlit UI at sandbox/app.py, HF Spaces-compatible

No candidate data ever leaves the machine during ranking. Honeypot detection via internal cross-checks (expert+duration mismatch, years implausibility, endorsement patterns) — no external API calls.